

VULNERABILITY ASSESSMENT AND PENETRATION TESTING (VAPT) REPORT

Web Application Security Assessment

Target: Damn Vulnerable Web Application (DVWA)

Prepared by	Himanshu
Course	Bachelor of Cyber Security
Assessment	Internship / Final-Year Project
Date	1 September 2026

Disclaimer

This assessment was carried out in a fully controlled and legal lab environment. The target application, DVWA (Damn Vulnerable Web Application), is an intentionally vulnerable application built for security training. It was installed locally on my own machine using XAMPP and was never exposed to the internet or any public network.

No real systems, live websites, or third-party data were touched at any point. All testing was performed only against my own local setup, purely for learning and academic purposes as part of my Bachelor of Cyber Security coursework.

Table of Contents

Disclaimer 2

Table of Contents..... 3

Executive Summary 4

Scope and Objectives..... 5

Test Environment..... 6

Methodology 8

Detailed Findings 9

1. SQL Injection 10

2. Reflected Cross-Site Scripting (XSS) 12

3. Command Injection (Remote Code Execution)..... 14

4. Unrestricted File Upload..... 16

Risk Summary 18

Conclusion..... 18

References 18

Executive Summary

This report presents the results of a Vulnerability Assessment and Penetration Test (VAPT) performed on a web application. The goal was to find security weaknesses in the application the same way a real attacker would, confirm that they work, and then explain how each one can be fixed.

The testing was done against DVWA, a deliberately vulnerable web app, running on a local server. Testing focused on the most common and most dangerous web vulnerabilities listed in the OWASP Top 10.

In total, four serious vulnerabilities were found and successfully exploited. Three of them are Critical because they can lead to complete control of the server or full theft of the database. One is High because it allows attackers to run malicious scripts in other users' browsers.

Summary of findings:

#	Vulnerability	Severity	Status
1	SQL Injection	Critical	Confirmed
2	Reflected Cross-Site Scripting (XSS)	High	Confirmed
3	Command Injection (RCE)	Critical	Confirmed
4	Unrestricted File Upload	Critical	Confirmed

Overall, the application is highly insecure. Because user input is trusted blindly and never checked, an attacker could steal all user data, take over the server, and run any command they want. The good news is that every issue found has a well-known, simple fix, which is covered in each section.

Scope and Objectives

Objective

The main objective of this assessment was to identify, exploit, and document security vulnerabilities in a web application, and to demonstrate the real-world impact of each issue with clear proof (screenshots).

In Scope

- The DVWA web application running locally at <http://localhost:8080/dvwa>
- Testing of common OWASP Top 10 web vulnerabilities
- Manual exploitation and evidence collection

Out of Scope

- Any live, public, or third-party website (no external systems were tested)
- Network-level, physical, or social-engineering attacks
- Denial-of-Service (DoS) testing

Rules of Engagement

All testing was performed only on my own locally hosted lab. The DVWA security level was set to "Low" so that the vulnerabilities could be clearly demonstrated for learning purposes. No automated mass-scanning was run against any external target.

Test Environment

The lab was set up on a local Windows machine. The table below lists the tools and setup used during the assessment.

Operating System	Windows (64-bit)
Web Server	Apache (via XAMPP 8.2.12), running on port 8080
Database	MariaDB / MySQL (port 3306)
Language	PHP 8.2.12
Target Application	DVWA (Damn Vulnerable Web Application)
Security Level	Low
Browser	Google Chrome
Testing Type	Manual black-box / grey-box testing

Environment setup → XAMPP running with Apache and MySQL active:

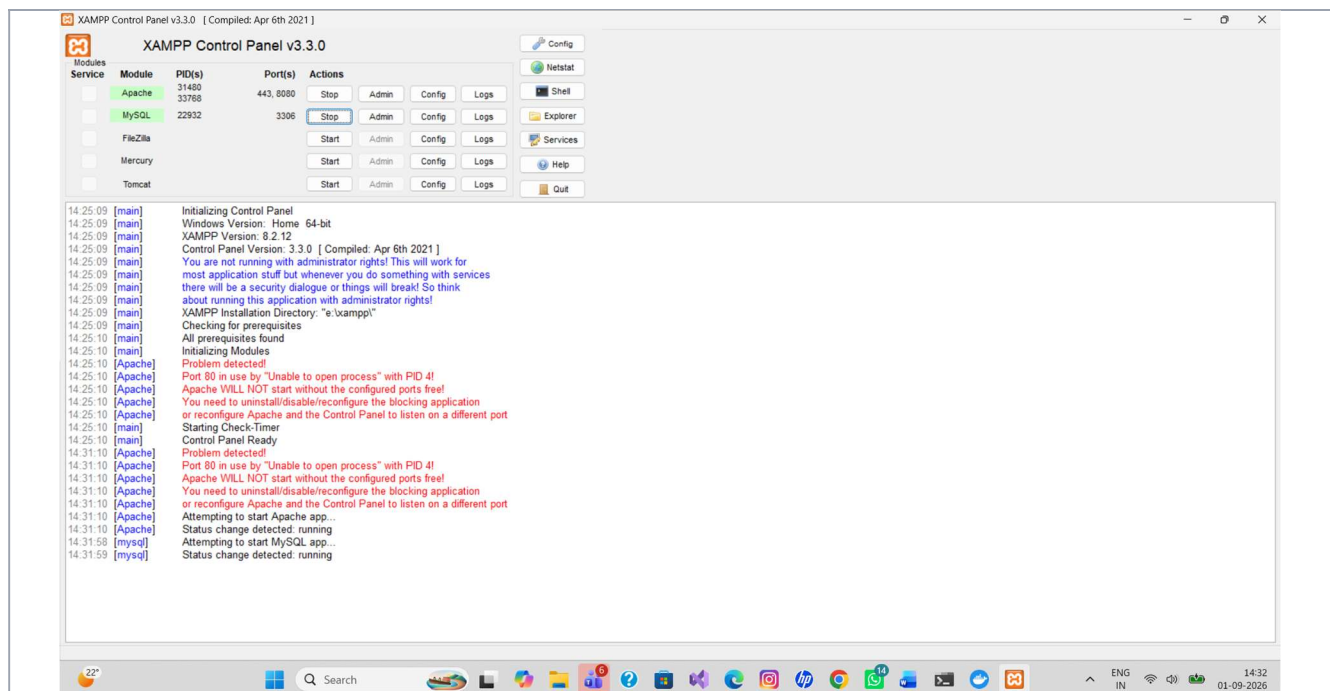


Figure: XAMPP Control Panel with Apache and MySQL running (green)

DVWA security level confirmed as "Low" before testing:

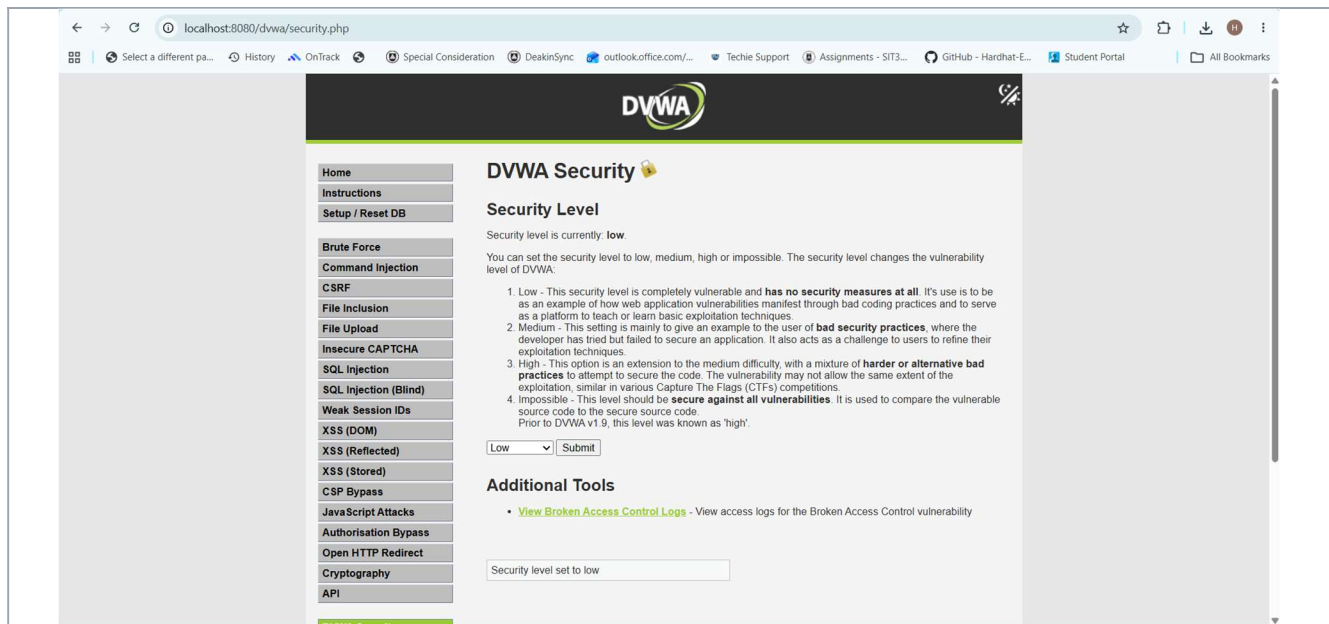


Figure: DVWA Security page showing 'Security level set to low'

Methodology

The assessment followed a simple, structured approach based on standard penetration-testing methodology (aligned with OWASP guidelines). For each vulnerability, the same repeatable process was used:

- Baseline: First, test the feature with normal input to understand how it is supposed to work.
- Test: Send crafted (malicious) input to see if the application handles it safely.
- Exploit: If it is vulnerable, exploit it to prove real impact (for example, dumping data or running commands).
- Evidence: Capture screenshots of both the normal behaviour and the successful exploit.
- Remediation: Note down how the issue can be fixed.

This baseline-then-exploit approach makes it easy to clearly see the difference between safe behaviour and a successful attack.

Detailed Findings

This section describes each vulnerability that was found and exploited, along with the evidence, its impact, and how to fix it.

1. SQL Injection

Severity	Critical
CVSS (approx.)	9.8 (Critical)
OWASP Category	A03:2021 – Injection
Affected Area	User ID field – /vulnerabilities/sqli/

What is this vulnerability?

SQL Injection happens when an application takes user input and puts it directly into a database query without checking it. This lets an attacker change the meaning of the query. Instead of just asking for one record, the attacker can trick the database into returning all records, or even the usernames and passwords of every user.

How I tested it

The SQL Injection page has a "User ID" box that is supposed to return the details of a single user. I tested it in three stages:

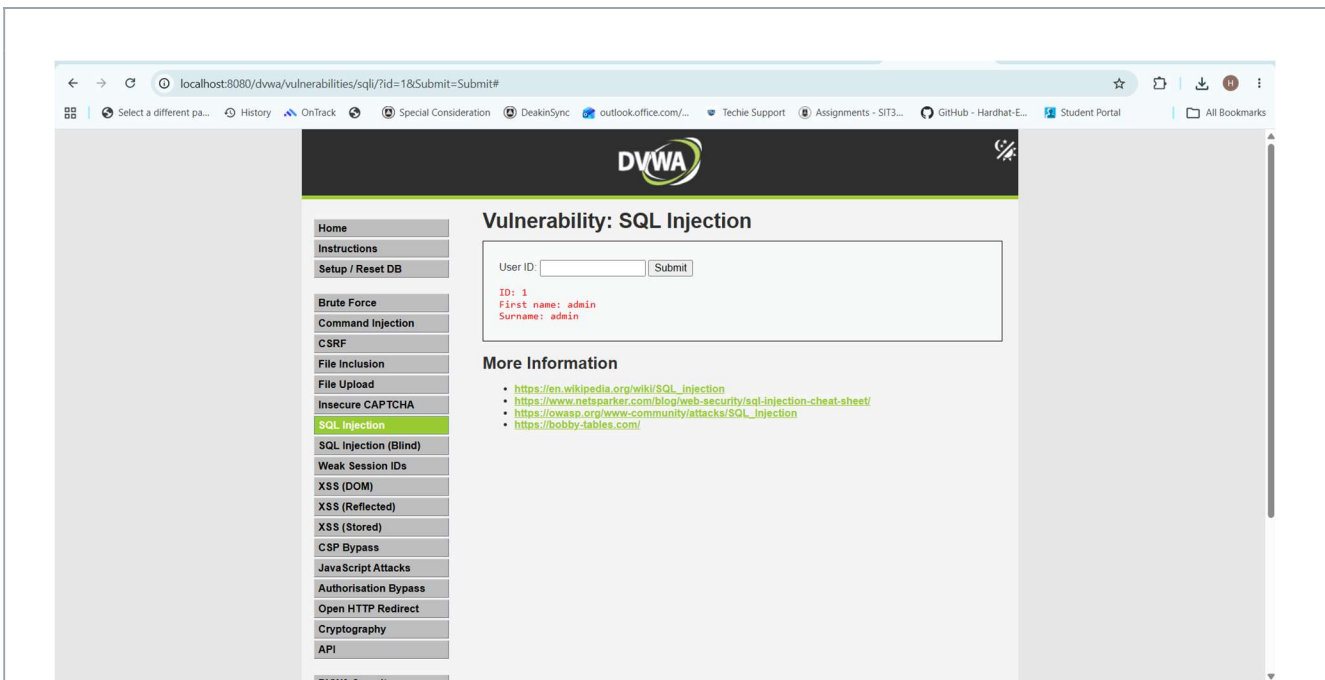
- Entered a normal value (1) to see the expected result for one user.
- Entered a classic injection to check if the input was being trusted blindly.
- Used a UNION-based query to pull out every username and password from the database.

Payloads used:

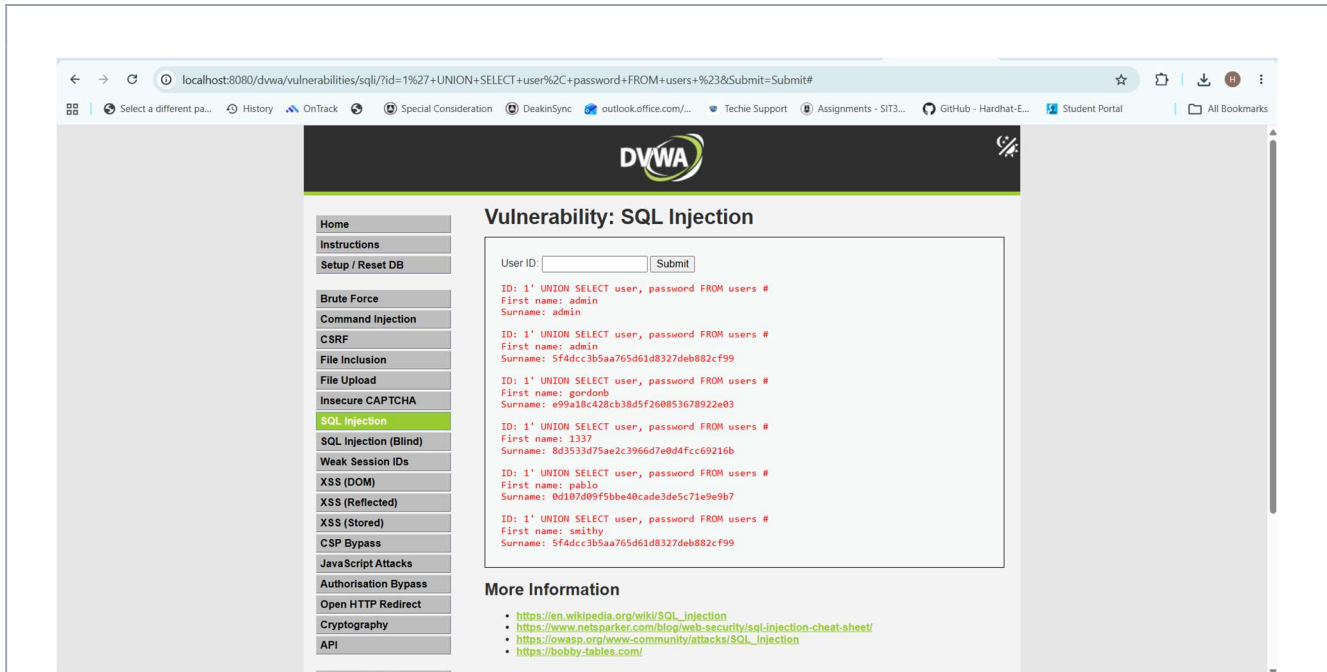
```
1' OR '1'='1
1' UNION SELECT user, password FROM users #
```

Evidence

Baseline (normal input) — how the feature behaves before any attack:



Exploit (attack input) — proof that the vulnerability works:



What happened / Result

When I entered 1, the app correctly returned just one user (admin). But when I entered `1' OR '1'='1`, the app returned ALL five users instead of one, which confirmed the input was not being checked. Finally, the UNION query returned the username and password hash of every user in the database (admin, gordonb, 1337, pablo, smithy). This is a full database breach. Two accounts even shared the same hash, and that hash (5f4dcc3b5aa765d61d8327deb882cf99) is the well-known MD5 for the word "password", showing weak, unsalted passwords make things even worse.

Impact — why it matters

This is one of the most dangerous web vulnerabilities. An attacker can steal the entire user database, including login credentials. From there they could log in as any user (including admins), access private data, or even modify or delete the database completely. For any real business, this would be a serious data breach.

How to fix it (Remediation)

- Use parameterised queries (prepared statements) so user input is never mixed directly into SQL code.
- Validate and sanitise all user input on the server side.
- Apply the principle of least privilege to the database account the app uses.
- Store passwords using a strong, salted hashing algorithm (like bcrypt), never plain MD5.

2. Reflected Cross-Site Scripting (XSS)

Severity	High
CVSS (approx.)	7.4 (High)
OWASP Category	A03:2021 - Injection (XSS)
Affected Area	Name field - /vulnerabilities/xss_r/

What is this vulnerability?

Cross-Site Scripting (XSS) happens when an application takes user input and shows it back on the page without cleaning it first. If the input contains JavaScript, the browser will run it. This means an attacker can inject scripts that run in another user's browser.

How I tested it

The Reflected XSS page has a "What's your name?" box that echoes whatever you type back onto the page. I tested it in two stages:

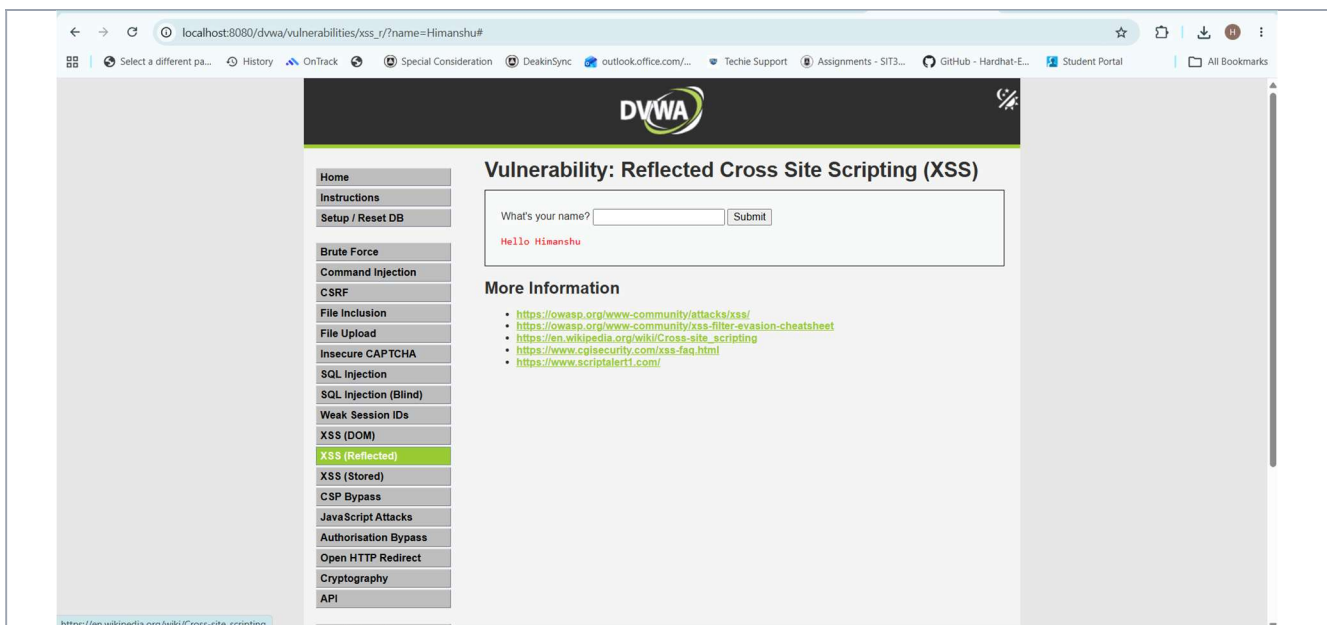
- Entered a normal name (Himanshu) to confirm the input is reflected on the page.
- Entered a JavaScript payload to check if the script would run.

Payloads used:

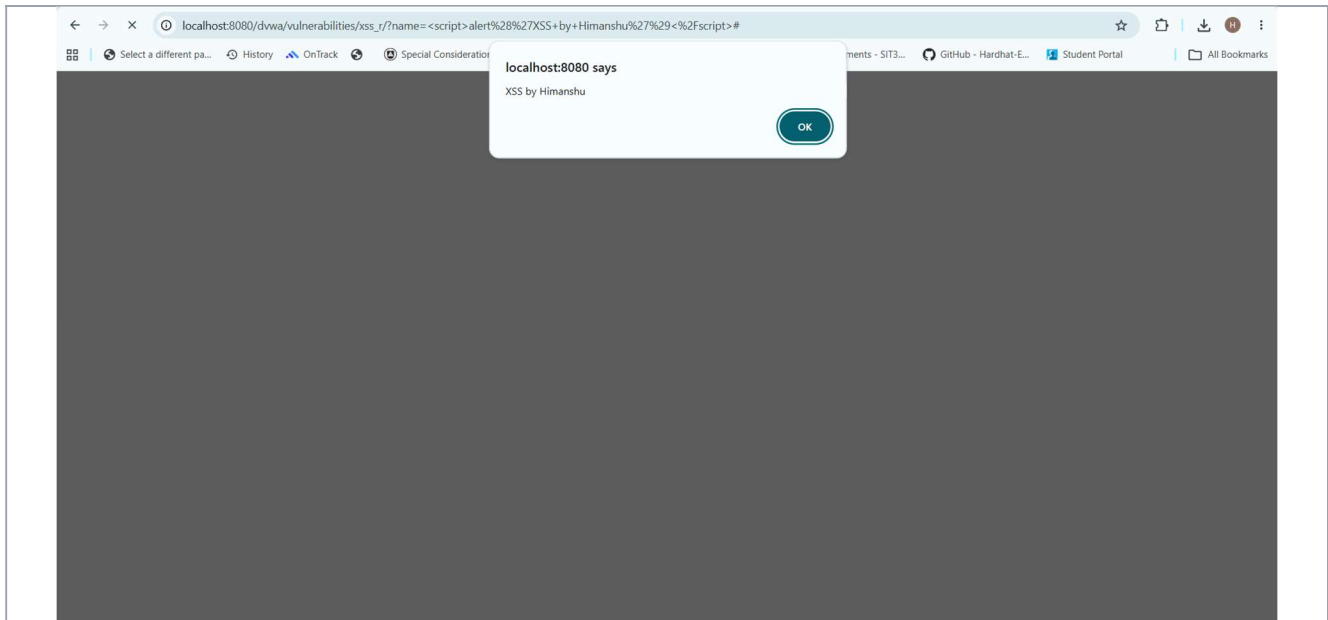
```
<script>alert('XSS by Himanshu')</script>
```

Evidence

Baseline (normal input) — how the feature behaves before any attack:



Exploit (attack input) → proof that the vulnerability works:



What happened / Result

Entering my name simply showed "Hello Himanshu" on the page, confirming the input is reflected. When I entered the script payload, the browser executed it and showed a pop-up box saying "XSS by Himanshu". This proves the application runs any script an attacker injects, instead of treating it as plain text.

Impact — why it matters

An attacker can craft a malicious link and trick a victim into clicking it. The injected script then runs in the victim's browser. This can be used to steal session cookies (and hijack their logged-in account), redirect them to fake pages, capture keystrokes, or display fake login forms to steal passwords.

How to fix it (Remediation)

- Encode all output so that user input is displayed as text, not run as code (output encoding).
- Validate and sanitise input on the server side.
- Use a Content Security Policy (CSP) to limit what scripts a browser is allowed to run.
- Set cookies with the HttpOnly flag so scripts cannot read them.

3. Command Injection (Remote Code Execution)

Severity	Critical
CVSS (approx.)	9.8 (Critical)
OWASP Category	A03:2021 - Injection
Affected Area	IP address field – /vulnerabilities/exec/

What is this vulnerability?

Command Injection happens when an application passes user input directly to the operating system to run a command, without checking it. This lets an attacker attach their own commands to the input, which the server then runs. This is extremely serious because it can give an attacker control of the whole server.

How I tested it

The Command Injection page has a "Ping a device" box that is meant to ping an IP address. On Windows, commands can be chained together using the & symbol. I tested it in stages:

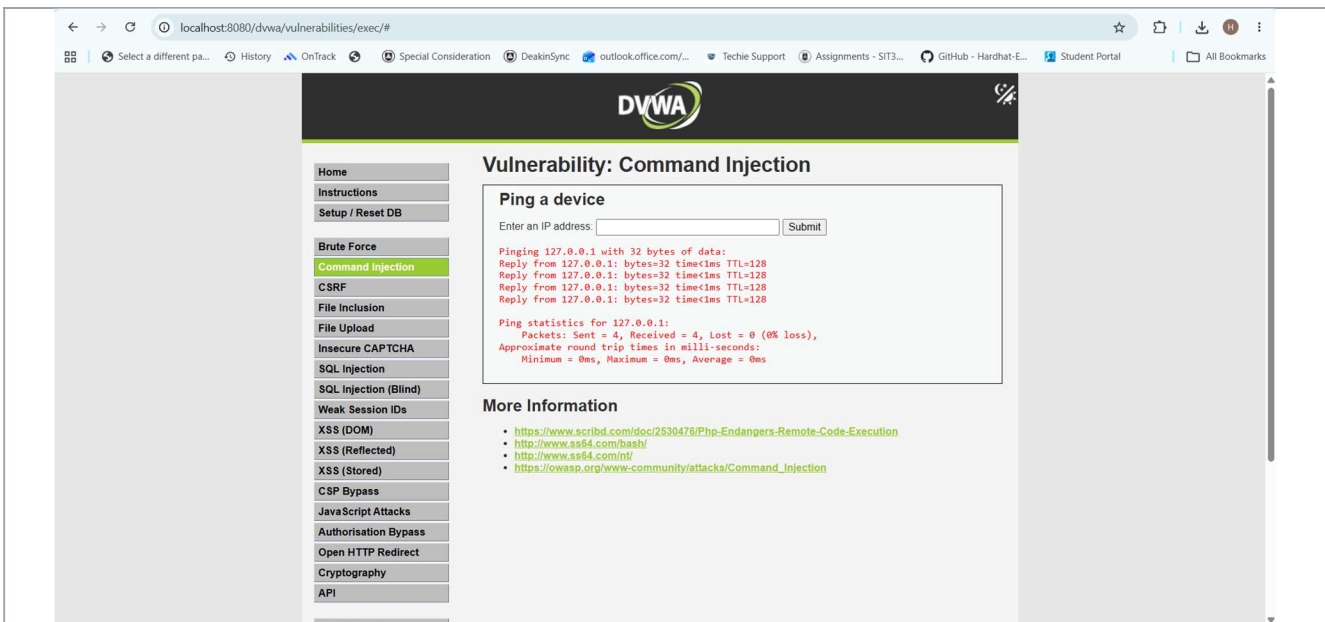
- Entered a normal IP (127.0.0.1) to confirm the ping feature works normally.
- Added an extra command after the IP to see if the server would run it.
- Listed the server's files to prove full command execution.

Payloads used:

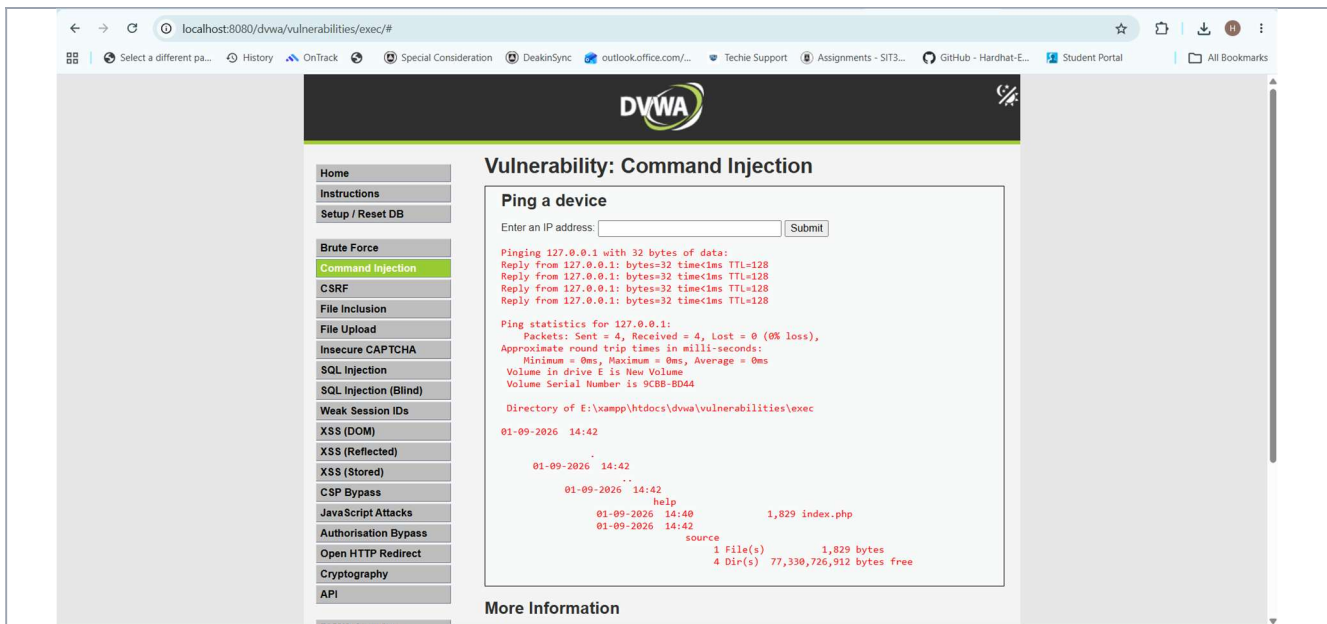
```
127.0.0.1 & whoami
127.0.0.1 & dir
```

Evidence

Baseline (normal input) - how the feature behaves before any attack:



Exploit (attack input) — proof that the vulnerability works:



What happened / Result

Entering 127.0.0.1 showed a normal ping result. When I added & whoami, the server also ran that command and returned its username (laptop-4l1tr3c3\hp). When I added & dir, the server returned a full listing of its own files and folders, including the exact path E:\xampp\htdocs\dvwa\vulnerabilities\exec. This proves I could run any operating-system command on the server.

Impact - why it matters

This is a worst-case scenario. Being able to run commands on the server means an attacker effectively controls the machine. They could read or delete any file, install malware or ransomware, create backdoor accounts, or use the server to attack other systems. For a real business this would mean a complete system compromise.

How to fix it (Remediation)

- Never pass user input directly to system commands.
- Use safe, built-in functions or libraries instead of calling the operating system shell.
- Strictly validate input using an allow-list (for example, only allow a valid IP-address format).
- Run the web server with the lowest privileges possible so damage is limited.

4. Unrestricted File Upload

Severity	Critical
CVSS (approx.)	9.8 (Critical)
OWASP Category	A05:2021 - Security Misconfiguration / Unrestricted Upload
Affected Area	File upload form – /vulnerabilities/upload/

What is this vulnerability?

This vulnerability happens when an application lets users upload files without properly checking the file type. The upload feature is supposed to accept only images, but if it does not check properly, an attacker can upload a malicious code file (a "web shell") and then run it on the server.

How I tested it

The File Upload page asks the user to "Choose an image to upload". Instead of an image, I uploaded a small PHP file (a web shell) that can run commands. I tested in two stages:

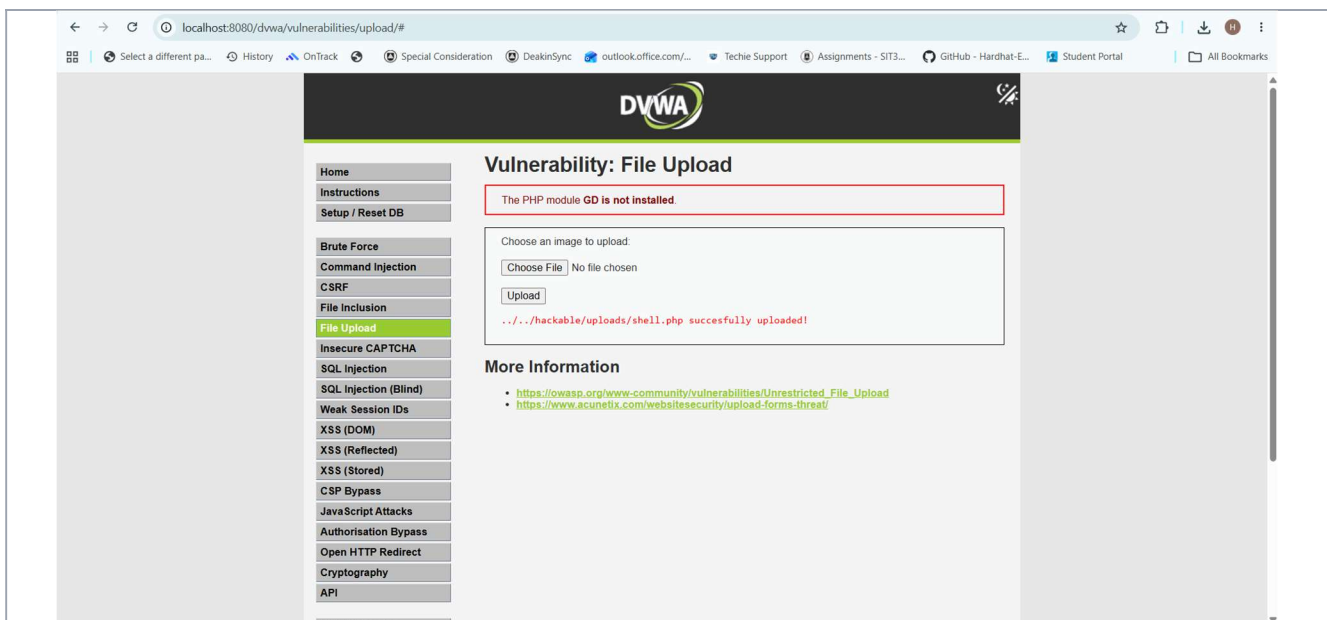
- Created a small PHP web-shell file that runs a command sent through the URL.
- Uploaded it through the image-upload form to see if it would be accepted.
- Opened the uploaded file directly in the browser to prove it runs on the server.

Payloads used:

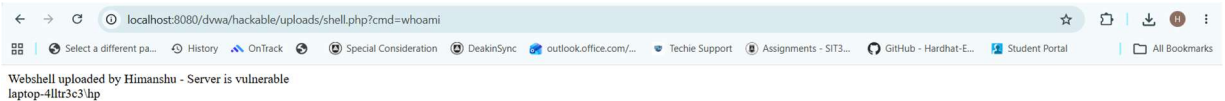
```
<?php echo "Webshell uploaded by Himanshu"; system($_GET['cmd']); ?>
http://localhost:8080/dvwa/hackable/uploads/shell.php?cmd=whoami
```

Evidence

Baseline (normal input) — how the feature behaves before any attack:



Exploit (attack input) - proof that the vulnerability works:



What happened / Result

The application accepted my PHP file even though it was supposed to accept only images, and confirmed: "../hackable/uploads/shell.php successfully uploaded!". I then opened the uploaded file in the browser and added ?cmd=whoami to the URL. The page displayed "Webshell uploaded by Himanshu" followed by the server username (laptop-4lltr3c3\hp). This proves my uploaded file actually runs on the server, giving remote control.

Impact → why it matters

Being able to upload and run code is essentially a full takeover of the server (remote code execution). An attacker with a web shell can browse files, steal data, run any command, install malware, and use the server as a base for further attacks. This is one of the most damaging vulnerabilities a web app can have.

How to fix it (Remediation)

- Only allow specific, safe file types and check the real file content, not just the file extension.
- Rename uploaded files and store them outside the web root so they cannot be run.
- Block execution of scripts in the upload folder (server configuration).
- Scan uploaded files and enforce size and type limits.

Risk Summary

The table below summarises all findings and their risk level. Critical issues should be fixed immediately, as they can lead to full data theft or complete server takeover.

#	Vulnerability	Severity	Status
1	SQL Injection	Critical	Confirmed
2	Reflected Cross-Site Scripting (XSS)	High	Confirmed
3	Command Injection (RCE)	Critical	Confirmed
4	Unrestricted File Upload	Critical	Confirmed

Conclusion

This assessment showed that the target web application is highly vulnerable. Four serious security issues were found and successfully exploited: SQL Injection, Reflected XSS, Command Injection, and Unrestricted File Upload. Three of these are Critical and one is High.

The root cause behind almost all these problems is the same simple mistake: the application trusts user input without ever checking or cleaning it. Because of this, an attacker could steal the entire database, run scripts in users' browsers, execute commands on the server, and even upload files to take full control.

The most important takeaway is that all these vulnerabilities have well-known, straightforward fixes. Validating and sanitising user input, using parameterised queries, encoding output, and properly checking uploaded files would prevent every issue found in this report. Following secure coding practices and the OWASP guidelines from the start is the best way to keep a web application safe.

This project gave me hands-on experience with the full penetration-testing process, from setting up a lab and exploiting real vulnerabilities to documenting the findings in a professional report.

References

- OWASP Top 10 → <https://owasp.org/www-project-top-ten/>
 - DVWA → Damn Vulnerable Web Application - <https://github.com/digininja/DVWA>
-